

Steam Game Recommendation Using Machine Learning: A Comparative Analysis of Recommendation Models

1st Mohammad Nahiyen Rahman

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh

Mohammad.nahiyen.rahman@g.bracu.ac.bd

2nd Ibrahim Faruk Bin Abul Kashem

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh

ibrahimfaruk.bin.abulkashem@g.bracu.ac.bd

3rd Hamim Farhat Dipu

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh

hamim.farhat.dipu@g.bracu.ac.bd

Group 03, CSE427 ,Machine Learning, Department of Computer Science and Engineering

Abstract—The large and diverse Steam game catalogue makes it difficult for players to discover games that match their interests, particularly when limited interaction data is available for new users or games. This project develops and evaluates a multi-signal recommendation system using the Kozyriev Game Recommendations on Steam dataset. The system combines playtime, explicit recommendation feedback, review helpfulness, and game metadata to model user preferences and address cold-start cases. Thirteen recommendation and ranking approaches were implemented and evaluated using a time-aware full-catalogue top-N evaluation protocol. Among the tested approaches, Implicit ALS achieved the strongest overall ranking performance, reaching approximately 0.0400 Recall@10 and 0.1218 NDCG@10, outperforming both popularity and truncated SVD baselines. The results also showed that confidence weighting based on hours played and review helpfulness improved recommendation quality, while personalized ranking provided substantially better catalogue coverage than popularity-based recommendations. Although cold-start games remained challenging on the global recommendation list, dedicated cold-start and new-release surfaces provided more effective alternatives. Overall, the findings demonstrate that combining multiple sources of user feedback can improve personalized game discovery in a sparse and popularity-skewed environment such as Steam.

Index Terms—recommender systems, collaborative filtering, implicit feedback, cold-start recommendation, matrix factorization, learning-to-rank, Steam, personalized recommendation

I. INTRODUCTION

A. Background

Steam has tens of thousands of games, with new ones added daily. With a catalogue that big, players can't just browse their way to games they'd enjoy and that's the exact problem recommender systems solve.

The most common approach, collaborative filtering (CF), looks at what users interacted with before and uses that pattern to guess what they'll like next. It needs no information about

the games themselves, just user activity, which is why it's so popular. But the catch is that CF struggles without much data. A brand-new game with only a few reviews, or a user who's barely used Steam, simply doesn't give the model enough to learn from. This is the "cold-start" problem, one of CF's biggest weaknesses.

Most Steam recommendation work tries to get around this with playtime, ownership data, or friend lists. But playtime doesn't always mean someone liked a game (they might have just left it running), and friend-based methods don't work at all if the dataset has no social data, which is the case here.

B. Motivation

Our dataset (Kozyriev's "Game Recommendations on Steam") gives us more than just playtime, an explicit recommend/not-recommend label, helpfulness votes on each review, and game tags. This let us try something different, instead of picking one signal, we combined hours played, the recommend label, and helpfulness into one model, and used game tags to help with games that don't have enough reviews yet.

We also stayed honest about what worked and what didn't, not every idea we tried helped, and we kept those results in the report too. We used machine learning because there's no simple rule for predicting which games someone will like; it's a pattern hidden across millions of reviews, exactly what ML is good at finding.

C. Problem Definition

Given the games a user has already liked, our task is to predict and rank the rest of the catalogue by how likely they are to enjoy each one next. More formally, for each user, we look at their past recommended games, then train a model that scores

every other game for that user. Sorting by this score should push the games they’ll actually like later near the top. Since about 86% of all reviews are "recommended," we treated this as a ranking problem rather than plain like/dislike classification, we care whether the right games show up near the top of a user’s list, not whether every label is classified correctly. We also split the data by time rather than randomly, training only on reviews before a set date and testing on reviews after it, so the model is judged on predicting the future, not memorizing shuffled data.

II. RELATED WORK

Our group split up and read nine papers on game recommendation, mostly Steam-specific projects plus a few newer ones (2023–2024) on cold-start problems. Here’s what we found and how it shaped our project.

A. Classical and Neural Collaborative Filtering

Cheuque et al. [3] found that deep models beat simpler ones on Steam data, but review sentiment barely helped, which is why we kept ALS as a serious baseline and stuck to structured features (hours, recommend label, helpfulness) instead of analyzing review text. Bunga et al. [2] turned playtime into star ratings and found SlopeOne/SVD++ worked well, but binning playtime loses detail, so we kept hours continuous instead. Batra et al. [1] combined playtime and the recommend label directly, closest in spirit to our approach, though they focused on error scores (RMSE) while we focused on ranking quality.

B. Deep Learning, Graphs, and Cold-Start

Wang et al. [7] used a deep autoencoder with rich user profiles and confirmed that more signals generally help, but our dataset lacks detailed profiles, so we enriched interaction data instead. Yang et al. [10] used a social graph neural network (SCGRec), which we couldn’t replicate since our dataset has no friend data; we used game tags as a substitute for social context. Yang et al. [9] tackled cold-start with graph pre-training, closest to our own cold-start goal, but since we have no graph, we mapped tags into our ALS model’s factor space instead, and tested cold items, cold users, and new releases separately.

C. Hybrid and Content-Based Approaches

Gong et al. [4] built a hand-weighted hybrid relying on friend data (which we don’t have); we kept their idea of blending signals but swapped in the recommend label and tested it against our trained model rather than assuming a hand-picked blend works. Salkanović et al. [6] emphasized diversity alongside accuracy, which pushed us to track catalogue coverage and popularity, not just Recall/NDCG. Wang et al. [8] built a similar hybrid but on a private dataset without formal statistical testing, we aimed for the same idea on a public dataset, backed by real significance tests.

D. Where Our Project Fits In

Across all nine papers, a few gaps kept showing up such as many strong methods need social data we don’t have, playtime-only ratings are rough approximations, diversity-focused models often trade off accuracy, evaluation is usually error-based rather than ranking-based, and cold-start is often treated as one issue instead of three (new item, new user, new release). Our project tries to close these gaps by combining multiple signals, using time-based evaluation, testing each cold-start case separately, and reporting what didn’t work just as honestly as what did.

III. DATASET DESCRIPTION

The project uses the publicly available Game Recommendations on Steam dataset published by Anton Kozyriev on Kaggle [5]. It was downloaded via the kagglehub API (with a local-folder fallback used when already present) and consists of four files: games.csv, users.csv, recommendations.csv, and games_metadata.json, totalling roughly 2.2 GB on disk.

TABLE I
OVERVIEW OF THE FOUR SOURCE FILES.

Table	Rows	Columns
games.csv	50,872	13
users.csv	14,306,064	3
recommendations.csv	41,154,794	8
games_metadata.json	50,872	5 (incl. tags)

This Table I summarises the raw table sizes as measured directly from the files.

Because recommendations.csv is too large to load into memory at once, it was scanned in 1-million-row chunks to compute exact full-file aggregates (row count, class balance, mean hours/helpful/funny, date range) and, separately, to draw a reproducible 2,000,000-row random sample used only for exploratory plotting – the models described in Section 4 are trained on a full-file dense subset, not this plotting sample.

TABLE II
KEY FEATURES USED FROM THE DATASET.

Feature	Type	Description
app_id	int	Unique game identifier, shared across all four tables.
user_id	int	Unique reviewer identifier.
is_recommended	bool	Binary label: whether the reviewer recommends the game.
hours	float	Total hours the reviewer played the game at review time.
helpful, funny	int	Helpfulness/funniness votes received by the review.
date	date	Review submission date; used to build the time-aware split.
title, date_release	str/date	Game title and release date.
rating	categorical	Steam review-based rating label (e.g. Positive, Very Positive, Mixed).
price_final, price_original, discount	float	Pricing fields (explored but not used as a primary ranker feature).
win, mac, linux, steam_deck	bool	Platform support flags.
tags	list[str]	Community-assigned tags per game, from the metadata file.
products, reviews	int	Per-user public profile counts (products owned, reviews written).

This Table lists the fields used from each table.

There is no single scalar prediction target in the classical sense; the task is a ranking problem. The binary is_recommended field defines which (user, game) interactions count as positive (like) evidence for training and as ground truth for evaluation, while hours and helpful are used as continuous confidence multipliers on those positive interactions rather than

as separate prediction targets. Rows with `is_recommended = False` are treated as explicit negative evidence and are used only in the dedicated explicit-negative experiments (Section 4), not as standard training positives.

On the full 41,154,794-row file, 35,304,398 rows (85.78%) are recommended and 5,850,396 (14.22%) are not, confirming a strongly imbalanced label, this is why the project frames evaluation around ranking metrics (Recall@K, NDCG@K) rather than balanced classification accuracy.

A direct sparsity measurement on the 2,000,000-row EDA sample gives a user–game interaction density of about 0.0045%, and an approximate full-file density (using $\text{lusers} \times \text{lgames}$) of about 0.0057% confirming that the interaction matrix is extremely sparse, which is the central motivation for combining collaborative filtering with a tag-based content pathway rather than relying on either alone.

IV. METHODOLOGY

A. Data Preprocessing

Preprocessing is implemented as two full-file passes over recommendations.csv, each streaming 1-million-row chunks so the 41.15-million-row file is never fully materialised in memory.

Pass 1 (count and filter). Every user and game is counted across the entire file. Constants fixed before this pass: `MIN_USER_REVIEWS = 10`, `MIN_GAME_REVIEWS = 30`, `MAX_USERS = 80,000`, `MAX_GAMES = 12,000`, `MAX_MODEL_ROWS = 4,000,000`, `TRAIN_TIME_FRAC = 0.80`, and `RANDOM_STATE = 42` for full reproducibility. Of 13,781,059 unique users and 37,610 unique games seen across the full file, 80,000 users and 12,000 games survived the minimum-review and metadata-intersection filters and were kept as the dense modelling population.

Pass 2 (materialise and split). The file is re-scanned, keeping only rows whose user and game both survived Pass 1, giving 4,830,132 dense rows. Since this exceeds `MAX_MODEL_ROWS`, users (not random rows) were down-sampled to 4,000,006 rows across 56,291 users, preserving each retained user’s full history rather than fragmenting it. A fixed calendar cutoff of 2021-11-02 (the 80th percentile of review dates) splits the table into 3,200,557 training rows (55,898 users, 10,738 games) and 799,449 test rows – a time-based split was used instead of a random row split because review volume is strongly non-stationary over time (Figure 11), and a random split would leak future reviews into training. The recommend rate is 81.47% in train and 81.72% in test, consistent with the full-file class balance.

Label and confidence construction. The training label is 1 for `is_recommended = True` rows. A continuous confidence weight is computed as:

$$c_{u,i} = \log(1 + \text{hours}_{u,i}) \times \log(1 + (1 + \text{helpful}_{u,i})) \quad (1)$$

so that both very long play sessions and highly-upvoted reviews increase a positive interaction’s weight in the implicit matrix, while the log transform curbs the influence of extreme

outliers (both hours and helpful votes are strongly right-skewed with many zeros, Figure 7). Rows with `is_recommended = False` (593,014 rows, 18.5% of the labelled train slice) are kept separately as explicit-negative evidence rather than discarded, and used only in the dedicated negative-feedback experiments described below.

Content vectorisation. A TF-IDF representation is built over the top-300 tags across all 50,872 metadata games, of which 10,738 overlap the train game index; games with an empty tag list receive an all-zero vector and cannot contribute to tag-overlap explanations, though they remain fully eligible for collaborative ranking. A sanity check (comparing cosine similarity for known-similar vs. known-dissimilar title pairs) confirmed the tag vectors meaningfully separate dissimilar games before they were trusted for content-based scoring.

Evaluation cohort. From the pool of eligible test users, 572 evaluation users were sampled (stratified across warm users with substantial training history and users with low pre-cutoff activity, target 300 of each), and this same fixed cohort is reused for every method throughout Section 5, which is what makes the paired significance tests possible.

B. Models

TABLE III
MODEL CHEAT-SHEET: CODES USED THROUGHOUT SECTION 5.

Code	Model
B1	Popularity (train like-count ranking)
B2	Content – mean tag profile per user, cosine ranking
B3	Truncated SVD (64 components) on the confidence-weighted matrix
B4	Hand-weighted hybrid blend of popularity, content, and SVD
B5	Hybrid blend with reserved cold-item slots
M1	Implicit Alternating Least Squares (ALS) – shipped ranker
M2a	LightGBM LambdaRank, tag-pool negative sampling
M2b	LightGBM LambdaRank, matched (uniform + popularity ^{0.75}) negatives
M2c	M2b + explicit-negative features and hard negatives
M2d	M2c + release-recency features
M3	Tag-to-ALS-factor cold-start map
M4	ALS with a tuned dislike-similarity penalty
–	Two-stage: tag retrieval shortlist re-ranked by M2b

Table III lists the model code used consistently throughout the report and results tables.

1) Baselines (B1–B3):

- B1 (Popularity): Ranks games by training-time positive interaction count. It serves as the baseline to beat and has near-zero long-tail coverage.
- B2 (Content): Represents users by the mean TF-IDF tag vector of their liked games and ranks candidates using cosine similarity. It requires no collaborative signal and is the only baseline usable for zero-interaction users with known tag preferences.
- B3 (Truncated SVD): Factorizes the sparse, confidence-weighted user–game matrix (55,898 × 10,738 with 2,607,542 non-zero entries) into 64 latent components, explaining ~26.4% of matrix variance. Games are ranked by the dot product of user and item factors.

2) *M1 – Implicit ALS (Shipped Model)*: The core collaborative model uses implicit.als.AlternatingLeastSquares with 64 latent factors, fit on the confidence-weighted positive matrix with $\alpha = 20.0$ and L_2 regularization $\lambda = 0.05$. ALS minimizes:

$$\min_{x_u, y_i} \sum_{u,i} c_{u,i} (p_{u,i} - x_u^T y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right) \quad (2)$$

where $p_{u,i} = 1$ for observed positives and $c_{u,i}$ is the confidence score. The model alternates closed-form updates of user factors x_u and item factors y_i . Running on the compiled implicit backend (with a NumPy fallback solver available) produced factor matrices shaped $(55,898 \times 64)$ and $(10,738 \times 64)$.

3) *M2 Family – LightGBM LambdaRank*: Four learning-to-rank variants share a core feature set (log popularity, user/item recommend rates, mean log hours/helpful, tag cosine, SVD/ALS scores, user activity stats) and a LightGBM LambdaRank objective, differing only in training data setup:

- M2a: Positives paired with content-similar tag-retrieval negatives. Uses 4,000 sampled users (max 20 positives, 40 negatives/user), creating a 237,166-row, 4,000-group matrix (77,166 positives).
- M2b: Splits training data into feature and label windows at the 80th-percentile date. Draws negatives from the full unseen catalog (50% uniform, 50% popularity^{0.75}), providing a fairer sampling setup for full-catalog ranking.
- M2c: Extends M2b by adding explicit dislike features (dislike-profile similarity, item dislike rate) and hard negatives from explicitly disliked games.
- M2d: Extends M2c with recency features (log release age, is-new-release flag) to test if recency improves general ranking alongside dedicated shelves.

Feature Importance: LightGBM split/gain reports (M2a) reveal tag cosine (64.1%) and ALS score (28.2%) dominate feature gain, with all other engineered features contributing under 3.5% combined. This shows the ranker relies almost entirely on the core signals from B2/M3 and M1.

4) *M3 – Tag-to-ALS-Factor Cold-Start Map*: M3 maps a game's TF-IDF tag vector to its ALS item factor using warm (non-cold) items. This allows cold items lacking interaction history to be mapped directly into the ALS latent space and scored against user factors like M1.

5) *M4 – ALS with Dislike Penalty*: M4 subtracts a tuned λ -weighted similarity to explicitly disliked items from the M1 ALS score. Cross-validation yielded an optimal penalty weight of $\lambda \approx 0$, collapsing M4 into the standard M1 model. This is documented as a measured near-duplicate result rather than an architectural gain.

6) *Hybrids (B4, B5) & Two-Stage*:

- B4: Blends popularity, content, and SVD scores via hand-tuned weights (calibrated separately for warm vs. cold items).
- B5: Extends B4 by reserving fixed top- N slots exclusively for cold items.

- Two-Stage: Retrieves a tag-similar shortlist, then re-ranks via M2b.

None of these three approaches outperformed the shipped ALS ranker on primary evaluation metrics.

7) *Cold-Start User Fold-in*: To serve live users missing from the training matrix, the production app executes an exact iALS fold-in solve with frozen M1 item factors (Y):

$$x_u = \left(Y^T Y + \sum_{i \in I_u} (c_i - 1) y_i y_i^T + \lambda I \right)^{-1} \sum_{i \in I_u} c_i y_i \quad (3)$$

using $\alpha = 20.0$ and $\lambda = 0.05$. This powers the live "For You" onboarding shelf using only initial user likes.

C. Tools Used

The pipeline is implemented in Python via a Kaggle notebook (notebook15d1e97395.ipynb, 89 cells: 31 markdown, 58 code).

- Data & Processing: pandas and NumPy for data chunking and dense arrays; SciPy for sparse matrices and Wilcoxon signed-rank tests.
- Modeling: scikit-learn (TruncatedSVD, HistGradientBoosting fallback); implicit for binary-compiled ALS; LightGBM (LGBMRanker).
- Pipeline & Utilities: matplotlib & seaborn for plots; tqdm for progress tracking; kagglehub for data retrieval.
- Reproducibility: Globally fixed RANDOM_STATE = 42.

Production artifacts (item factors, tag matrix, popularity lists, LambdaRank model) export via .npy, .npz, CSV, and JSON formats, enabling lightweight web app serving without notebook or 41-million-row dataset dependencies. Offline testing verified four core recommendation shelves: Popular now, For you, Hidden gems, and New releases.

V. RESULTS AND EVALUATION

A. Evaluation Metrics

All 13 methods are evaluated under an identical full-catalogue protocol: ranking all 10,738 unseen training games for 572 evaluation users. Recommendations at top $K = 10$ (with $K = 50, 100$ for recall) are scored against held-out test items using the following metrics:

- Recall@K:

$$\text{Recall@K} = \frac{|\text{recommended}_{1:K} \cap \text{relevant}|}{|\text{relevant}|} \quad (4)$$

- NDCG@K:

$$\text{NDCG@K} = \frac{\sum_{i=1}^K \frac{\mathbb{1}[\text{item}_i \in \text{relevant}]}{\log_2(i+1)}}{\sum_{i=1}^{\min(|\text{relevant}|, K)} \frac{1}{\log_2(i+1)}} \quad (5)$$

- HitRate@K: Fraction of users receiving at least one relevant recommendation in their top- K list.
- Catalogue Coverage: Fraction of the 10,738 games surfaced across all top- K lists, serving as a proxy for long-tail exposure.

- Average Popularity Percentile: Mean popularity rank of recommendations (higher values reflect blockbuster bias; lower values show long-tail reach).
- Cold / Warm HitRate@10: HitRate@10 split by item interaction history (low vs. high) to isolate cold-start discovery performance.

Validation & Statistical Testing

- Chance Floor: Analytic chance for Recall@10 across the 10,738-game catalogue is 10/10,738 $\approx$ 0.00093. Integrity checks (Cell 62) confirmed random rankings match this floor while non-random rankers perform far above it.
- Significance Tests: Paired tests across the 572 users evaluate metric differences using a 10,000-resample bootstrap confidence interval, a Wilcoxon signed-rank test, and Holm–Bonferroni corrections across all comparisons.

B. Results

Table IV reports the consolidated headline comparison of all thirteen methods under the full-catalogue protocol (572 evaluation users, K=10), which is the primary results table for this project.

TABLE IV
FULL-CATALOGUE EVALUATION, ALL 13 METHODS, 572 USERS, K = 10.

Model	Recall@10	NDCG@10	HitRate@10	Coverage	Avg. Pop. Pct.
M3 Tag→factor cold-start	0.0400	0.1218	0.4685	0.1160	0.9607
M1 ALS (shipped)	0.0400	0.1218	0.4685	0.1155	0.9629
M4 ALS + dislike penalty	0.0400	0.1216	0.4685	0.1148	0.9635
M2a LTR (tag-pool negatives)	0.0382	0.1073	0.4580	0.0965	0.9496
M2b LTR (matched negatives)	0.0347	0.0920	0.4476	0.1450	0.8230
M2d LTR + recency	0.0326	0.0857	0.4266	0.1284	0.8376
M2c LTR + explicit negatives	0.0323	0.0912	0.4371	0.1386	0.8814
B3 MF (SVD)	0.0281	0.0965	0.4196	0.0306	0.9858
B1 Popularity	0.0246	0.0767	0.3969	0.0022	0.9995
Two-stage (tags → M2b)	0.0204	0.0521	0.2570	0.1850	0.8051
B4 Hybrid	0.0193	0.0623	0.3199	0.0327	0.9831
B5 Hybrid + Reserved	0.0189	0.0610	0.3094	0.0350	0.9581
B2 Content (tags)	0.0095	0.0326	0.1573	0.2018	0.7332

Cold-item HitRate@10 is $\approx$ 0 for every method except M2b/M2c/Two-stage/B2, which reach at most 0.0047 – confirming that, on the main list, cold items essentially never surface among the top 10 regardless of method, which motivated the dedicated cold-only surface reported below.

C. Visualizations

Fig. 1 plots Recall@10, cold-item HitRate@10, and average popularity percentile side by side for all methods on the main evaluation list.

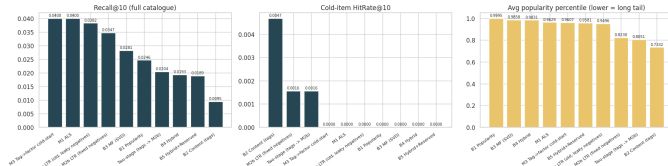


Fig. 1. Main-list Recall@10, cold-item HitRate@10, and average popularity percentile for all evaluated methods (572 users, full catalogue). M1 ALS and M3 lead on accuracy; only a handful of methods reach any cold items at all on this list.

Fig. 2 shows the dedicated cold-items-only ranking task, in which candidates are restricted to the 2,761 cold games so that a method is not forced to outrank thousands of popular titles to succeed.

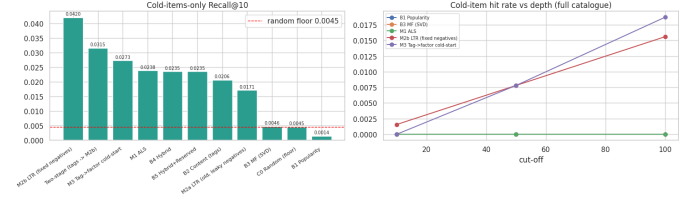


Fig. 2. Cold-items-only ranking surface (2,761 cold candidates, 222 of 572 users have a relevant cold item, chance Recall@10 $\approx$ 0.0036). M2b LTR is the strongest method here at Recall@10 $\approx$ 0.042, roughly 9 $\times$ the measured random floor of 0.0045.

Fig. 3 shows the bootstrap confidence interval for the rarer event of a cold item appearing deeper in the main list (top 50/100), supporting the reading that the main top-10 list is simply the wrong place to look for cold-item success.

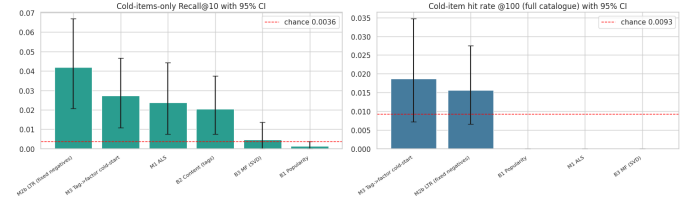


Fig. 3. Cluster-bootstrap confidence interval on item-level cold-hit rate at depth, resampled over users.

Fig. 4 shows the cold-start-user onboarding curve: Recall@10 for ALS/M3 fold-in and content-only scoring as a function of the number of seed likes a brand-new user has provided, compared against the flat popularity baseline.

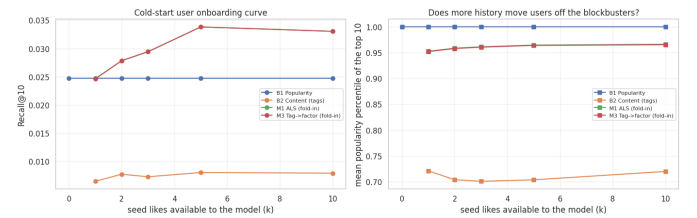


Fig. 4. Onboarding curve: Recall@10 vs. number of seed likes ($k = 0, 1, 2, 3, 5, 10$), 566 simulated new users. ALS/M3 fold-in overtakes the flat popularity baseline from $k = 5$ seed likes onward and remains ahead at $k = 10$; tag-cosine content scoring never overtakes popularity within $k \leq 10$.

D. Model Comparison

Paired significance testing (10,000-resample bootstrap + Wilcoxon + Holm correction, same 572 users) confirms that the key gaps in Table IV are meaningful:

- M1 ALS vs. B3 SVD: Recall@10 +0.0119 [95% CI +0.0045, +0.0204], Holm p=0.006; NDCG@10 +0.0254 [+0.0138, +0.0377], Holm p=0.0004 — significant.

- M1 ALS vs. B1 Popularity: Recall@10 +0.0153 [+0.0070, +0.0244], Holm $p \approx 2 \times 10^{-5}$; NDCG@10 +0.0452 [+0.0313, +0.0594], Holm $p \approx 5 \times 10^{-8}$ — significant.
- M1 ALS vs. M2b LTR: NDCG@10 is ≈ 0.030 higher for M1; the Recall@10 difference is not always significant.
- M2a vs. M2b: M2a achieves higher Recall@10 (0.0382 vs. 0.0347), showing that tag-pool negatives did not underperform the more complex matched-negative approach.

For the final ranking decision, M1 ALS also outperformed M2c: M2c was -0.0077 in Recall@10 (Holm $p=0.078$) and -0.0306 in NDCG@10 (Holm $p \approx 6 \times 10^{-6}$). With similar engineering cost and a simpler fold-in process for cold-start users, M1 Implicit ALS was selected as the main “For you” ranker.

For the new-release shelf, M1 ALS achieved Recall@10 = 0.1603, compared with 0.0222 for newest-first sorting across 1,588 games released within 365 days. Personalised ranking therefore clearly outperformed date sorting, although only 762 of the 1,588 titles were cold by interaction count, so this mainly reflects recent games with existing history.

E. Analysis

Three main findings emerge.

Confidence weighting helps, but mainly when combined multiplicatively. In a controlled ablation, hours $\times$ helpfulness achieved Recall@10 = 0.0400, compared with 0.0377 for binary labels, 0.0363 for hours-only, 0.0376 for helpfulness-only, and 0.0348 for additive hours+0.3 $\times$ helpfulness. The multiplicative formula was the only variant that clearly improved over the binary baseline.

Explicit negative feedback did not improve ranking. M4 was statistically indistinguishable from M1 (-0.00001 Recall@10, CI includes zero), while M2c did not reliably beat M2b (-0.0024 Recall@10, CI includes zero). Although all 593,014 explicit-negative rows were correctly incorporated, both model families suggest that the positive interaction matrix already captures most of their useful information.

Cold-start is three different problems. Cold items rarely appear in the main top-10 list (HitRate@10 ≈ 0). When ranking only cold items, M2b reaches Recall@10 ≈ 0.042 , about 9 $\times$ the random floor, suggesting that cold inventory is rankable when separated from popular titles. This supports a dedicated “Hidden gems” shelf. New releases behave differently: personalised ranking beats date sorting by roughly 7 $\times$ because many recent games already have review history. For cold-start users, fold-in personalisation reliably beats popularity only after about five seed likes (Fig. 4), suggesting that new users should initially receive popularity-based recommendations.

Finally, there were no signs of gross overfitting under this evaluation protocol. The time-based split kept the 572 users’ relevant test-period games unseen during training, while random-ranker, fixed-ranker, and deterministic repeat-scoring checks all passed before the results were accepted.

VI. DISCUSSION

A. Interpretation of Findings

The project’s seven problem statements were checked against the computed results rather than assumed. Six of the seven are supported to some degree: personalised ranking beats popularity; hours and helpfulness add useful preference information; cold-start games are rankable on a dedicated surface; new releases benefit from personalisation; cold-start users are supported from five seed likes onward; and long-tail discovery improves, with catalogue coverage of 0.116 vs. 0.002 for popularity. Explicit negative feedback was successfully incorporated and measured, but produced no ranking gain, making it the project’s main negative result.

The results also show that M1 Implicit ALS, despite being a simple 64-factor model, matches or outperforms the more complex alternatives, including four LambdaRank variants, two hand-built hybrids, and a two-stage retrieve-and-rerank pipeline. Over 92% of LightGBM’s total gain comes from tag cosine and the ALS score, suggesting that the complex models largely reproduce signals already captured by simpler methods. This highlights an important lesson: model complexity should be justified by statistical improvement, not architecture alone.

B. Limitations

Several limitations affect how broadly these results can be interpreted:

- Model scope: No deep, autoencoder, graph, or social models were implemented, so the project cannot claim to outperform those approaches. This reflects project scope and reproducibility constraints, not evidence that they would perform worse.
- Dense-subset conditioning: Results apply to users with at least 10 reviews and games with at least 30, capped at 80,000 users / 12,000 games before further downsampling. Very sparse users and games are excluded.
- Cold items: Main-list cold-item HitRate@10 is 0.0000, so cold inventory requires a dedicated surface.
- New releases: Only 762 of 1,588 new titles are also cold, meaning the new-release evaluation is not a pure zero-history test.
- Negative feedback: Explicit negatives did not improve ranking despite being tested in two model families.
- Tag coverage: Some games, including blockbusters, have empty tag lists, limiting “because: shared tags” explanations.
- Hybrid models: B4, B5, and the two-stage tag pipeline all underperformed M1 ALS, suggesting hybridisation is more useful for shelf design, such as “Hidden gems,” than as the main ranker.
- Deployment scope: LambdaRank models were exported but not used in the live demonstration; the validated deployment path uses ALS/M3.

C. Possible Improvements

Several practical improvements could strengthen the system. First, the confidence-weighting ablation could test additional functions, such as a saturating transformation for extreme hours, to determine whether the multi-signal improvement can be increased.

Second, the cold-item and new-release shelves could be evaluated with the same paired bootstrap procedure used for the main list, providing stronger confidence intervals for the smaller cohorts of 222 and 492 users, respectively.

Third, empty tag vectors could be enriched using the game descriptions already available in `games_metadata.json`, which were not used for content features in this run.

For longer-term work, implementing a deep or graph-neural collaborative baseline would provide a stronger accuracy comparison. If a dataset containing friend-graph information becomes available, the social-aware component used by SCGRec could also be revisited.

VII. CONCLUSION AND FUTURE WORK

A. Summary

This project developed a personalised Steam game recommendation system designed to handle information overload and cold-start challenges using the public Kozyriev dataset (41,154,794 recommendation rows, 50,872 games, 14,306,064 users). A reproducible two-pass preprocessing pipeline produced a time-split modelling table with 3,200,557 training rows, 55,898 users, and 10,738 games, using a 2021-11-02 cutoff.

Thirteen ranking methods were evaluated under the same full-catalogue top-N protocol on 572 held-out users. The final model, Implicit ALS with hours $\times$ helpfulness confidence weighting, achieved Recall@10 $\approx$ 0.0400 and NDCG@10 $\approx$ 0.1218, significantly outperforming both truncated SVD and popularity ranking based on paired bootstrap, Wilcoxon, and Holm-corrected tests.

Dedicated evaluations also showed that cold-start items are better handled through separate surfaces, personalised new-release recommendations outperform date sorting, and user personalisation becomes useful from five seed likes onward. Explicit negative feedback, despite being correctly modelled in two ways, produced no reliable ranking improvement. Finally, the exported serving artifacts reproduced the notebook’s recommendation shelves exactly after reload.

B. Future Work

Future improvements should focus on four areas:

- **Advanced models:** Implement a deep or graph-based collaborative model, such as DeepFM or a GNN, under the same evaluation protocol to determine whether it can improve on ALS.
- **Better content features:** Use game descriptions already available in the metadata to improve tag-based recommendations and reduce empty “because: shared tags” explanations.

- **Stronger evaluation:** Apply the same paired bootstrap testing to the cold-item and new-release shelves for more reliable confidence intervals.
- **Social recommendation:** If a dataset with friend-graph information becomes available, evaluate social-aware personalisation such as SCGRec alongside the current multi-signal approach.

REFERENCES

- [1] Sahil Batra, Vibhu Sharma, Xin Wang, Yifan Wang, and Yuan Sun. Steam recommendation system: Using implicit and explicit indicators to generate game recommendations for users on steam, 2023.
- [2] Ricardo Bunga, Fernando Batista, and Ricardo Ribeiro. From implicit preferences to ratings: Video games recommendation based on collaborative filtering. In Proceedings of the 13th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management (IC3K/KDIR), pages 209–216, 2021.
- [3] Gustavo Cheuque, José Guzmán, and Denis Parra. Recommender systems for online video game platforms: the case of STEAM. In Companion Proceedings of The 2019 World Wide Web Conference (WWW ’19), pages 763–771, 2019.
- [4] Jiazhen Gong, Yishu Ye, and Kostas Stefanidis. A hybrid recommender system for steam games. In Information Search, Integration, and Personalization (ISIP 2019), volume 1197 of Communications in Computer and Information Science, pages 133–144. Springer, 2019.
- [5] Anton Kozyriev. Game recommendations on Steam. \url{https://www.kaggle.com/datasets/antonkozyriev/game-recommendations-on-steam}, 2022. Kaggle dataset.
- [6] Ervin Salkanović, Nedim Zukorlić, Lamija Oković, and Dino Kečo. A data analysis of steam’s game catalog and diverse recommendation strategies. International Journal of Computer Applications, 186(54):39–49, 2024.
- [7] Dylan Wang, Melody Moh, and Teng-Sheng Moh. Using deep learning and steam user data for better video game recommendations. In Proceedings of the 2020 ACM Southeast Conference (ACMSE), pages 154–159, 2020.
- [8] Shuo Wang, Yue Xu, Shuang Li, and Jing Li. Optimization of weighted hybrid algorithm based on collaborative filtering and content filtering in steam game recommendation. Journal of Computing and Electronic Information Management, 17(2):26–33, 2025.
- [9] Hao Yang, Ge Tian, Chen Xu, and Rui Wang. Algorithms for cold-start game recommendation based on GNN pre-training model. The Computer Journal, 67(9):2787–2798, 2024.
- [10] Liangwei Yang, Zhiwei Liu, Yu Wang, Chen Wang, Ziwei Fan, and Philip S. Yu. Large-scale personalized video game recommendation via social-aware contextualized graph neural network. In Proceedings of the ACM Web Conference 2022 (WWW ’22), pages 3376–3386, 2022.